

# CS50's Introduction to Databases with SQL

OpenCourseWare

Donate [🔗 \(https://cs50.harvard.edu/donate\)](https://cs50.harvard.edu/donate)

Carter Zenke (<https://carterzenke.me>)

[carter@cs50.harvard.edu](mailto:carter@cs50.harvard.edu)

[🐙 \(https://github.com/carterzenke\)](https://github.com/carterzenke) [in \(https://www.linkedin.com/in/carterzenke/\)](https://www.linkedin.com/in/carterzenke/)

David J. Malan (<https://cs.harvard.edu/malan/>)

[malan@harvard.edu](mailto:malan@harvard.edu)

[f \(https://www.facebook.com/dmalan\)](https://www.facebook.com/dmalan) [🐙 \(https://github.com/dmalan\)](https://github.com/dmalan) [@ \(https://www.instagram.com/davidjmalan/\)](https://www.instagram.com/davidjmalan/)

[in \(https://www.linkedin.com/in/malan/\)](https://www.linkedin.com/in/malan/) [👤 \(https://www.reddit.com/user/davidjmalan\)](https://www.reddit.com/user/davidjmalan)

[🗨️ \(https://www.threads.net/@davidjmalan\)](https://www.threads.net/@davidjmalan) [🐦 \(https://twitter.com/davidjmalan\)](https://twitter.com/davidjmalan)

[🐦 \(https://twitter.com/davidjmalan\)](https://twitter.com/davidjmalan)

## 36 Views



*Fine Wind, Clear Morning by Katsushika Hokusai*

## Problem to Solve

From 1830 to 1832, the Japanese artist Katsushika Hokusai created [36 woodblock prints](https://en.wikipedia.org/wiki/Thirty-six_Views_of_Mount_Fuji) ([https://en.wikipedia.org/wiki/Thirty-six\\_Views\\_of\\_Mount\\_Fuji](https://en.wikipedia.org/wiki/Thirty-six_Views_of_Mount_Fuji)) depicting 36 different views of Mount Fuji ([https://en.wikipedia.org/wiki/Mount\\_Fuji](https://en.wikipedia.org/wiki/Mount_Fuji)), a volcano on the [Honshū island](https://en.wikipedia.org/wiki/Honshu) (<https://en.wikipedia.org/wiki/Honshu>) of Japan. Among the series' most famous works are [Fine Wind, Clear Morning](https://en.wikipedia.org/wiki/Fine_Wind,_Clear_Morning) ([https://en.wikipedia.org/wiki/Fine\\_Wind,\\_Clear\\_Morning](https://en.wikipedia.org/wiki/Fine_Wind,_Clear_Morning)) and [The Great Wave off Kanagawa](https://en.wikipedia.org/wiki/The_Great_Wave_off_Kanagawa) ([https://en.wikipedia.org/wiki/The\\_Great\\_Wave\\_off\\_Kanagawa](https://en.wikipedia.org/wiki/The_Great_Wave_off_Kanagawa)). The prints became so influential that another Japanese artist of the time period—Utagawa Hiroshige—created [his own series of 36 prints](https://en.wikipedia.org/wiki/Thirty-six_Views_of_Mount_Fuji_(Hiroshige)) ([https://en.wikipedia.org/wiki/Thirty-six\\_Views\\_of\\_Mount\\_Fuji\\_\(Hiroshige\)](https://en.wikipedia.org/wiki/Thirty-six_Views_of_Mount_Fuji_(Hiroshige))), each depicting a new view of Fuji.

In `views.db`, you'll find details on the 36 prints created, respectively, by Hokusai and Hiroshige. In total, you'll have data on 72 prints. Prints like these tend to be appreciated for their aesthetics, not their statistics, but computer science now helps create additional *numeric* insights about artwork. In addition to each print's title and author, you'll find some statistics commonly used in [computational image analysis](https://en.wikipedia.org/wiki/Computer_vision) ([https://en.wikipedia.org/wiki/Computer\\_vision](https://en.wikipedia.org/wiki/Computer_vision)): the print's *average color*, its *brightness*, its *contrast*, and its *entropy*. In the accompanying `.sql` files, write SQL queries to answer questions about this database of 72 prints and the statistics about their composition.

► Curious about average color, brightness, contrast, or entropy?

## Demo

```
| Futami Bay in Ise Province
| Lake Suwa in Shinano Province
| Shiojiri Pass in Shinano Province
| Misaka Pass in Kai Province
| The Ōtsuki Plain in Kai Province
| Dog Eye Pass in Kai Province
| Kogane Plain in Shimōsa Province
| Kuroto Bay in Kazusa Province
| Mt. Kanō in Kazusa Province
| The Hota Coast in Awa Province
```

```
+-----+
sqlite> █
```

# Distribution Code

For this problem, you'll need to download `views.db`, along with several `.sql` files in which you'll write your queries.

## ▼ Download the distribution code

Log into [cs50.dev \(https://cs50.dev/\)](https://cs50.dev), click on your terminal window, and execute `cd` by itself. You should find that your terminal window's prompt resembles the below:

```
$
```

Next execute

```
wget https://cdn.cs50.net/sql/2024/x/psets/0/views.zip
```

in order to download a ZIP called `views.zip` into your codespace.

Then execute

```
unzip views.zip
```

to create a folder called `views`. You no longer need the ZIP file, so you can execute

```
rm views.zip
```

and respond with “y” followed by Enter at the prompt to remove the ZIP file you downloaded.

Now type

```
cd views
```

followed by Enter to move yourself into (i.e., open) that directory. Your prompt should now resemble the below.

```
views/ $
```

If all was successful, you should execute

```
ls
```

and see a database named `views.db` alongside several `.sql` files. Executing `sqlite3 views.db` should open the database in `sqlite3`, via which you'll execute SQL queries. If not, retrace your steps and see if you can determine where you went wrong!

## Schema

---

In `views.db` you'll find a single table, `views`. In the `views` table, you'll find the following columns:

- `id`, which uniquely identifies each row (print) in the table
- `print_number`, which identifies the number of the print in either Hokusai's or Hiroshige's series
- `english_title`, which is the English title of the print
- `japanese_title`, which is the Japanese title of the print
- `artist`, which is the last name of the print's artist
- `average_color`, which is the [hexadecimal representation](https://www.pluralsight.com/blog/tutorials/understanding-hexadecimal-colors-simple) (<https://www.pluralsight.com/blog/tutorials/understanding-hexadecimal-colors-simple>) of the color found by averaging the colors of each pixel in the image
- `brightness`, which is a number corresponding to the overall lightness or darkness of the image
- `contrast`, which is a number representing the extent of the difference between light and dark areas of the image
- `entropy`, which is a measure used to quantify the complexity of the artwork

## Specification

---

For each of the following questions, you should write a single SQL query that outputs the results specified by each problem. Your response must take the form of a single SQL query. You **should not** assume anything about the `id`s of any particular observations: your queries should be accurate even if the `id` of any particular observation were different. Finally, each query should return only the data necessary to answer the question.

1. In `1.sql`, write a SQL query that a translator might take interest in: list, side by side, the Japanese title and the English title for each print. Ensure the Japanese title is the first column, followed by the English title.
2. In `2.sql`, write a SQL query to list the average colors of prints by *Hokusai* that include "river" in the English title. (As an aside, do they have any hint of blue?)



3. In `3.sql`, write a SQL query to count how many prints by *Hokusai* include “Fuji” in the English title. Though all of Hokusai’s prints focused on Mt. Fuji, in how many did “Fuji” make it into the title?
4. In `4.sql`, write a SQL query to count how many prints by *Hiroshige* have English titles that refer to the “Eastern Capital”. Hiroshige’s prints were created in Japan’s “Edo period,” referencing the eastern capital city of [Edo \(https://en.wikipedia.org/wiki/Edo\)](https://en.wikipedia.org/wiki/Edo), now Tokyo.
5. In `5.sql`, write a SQL query to find the highest contrast value of prints by *Hokusai*. Name the column “Maximum Contrast”. Does Hokusai’s prints most contrasting print actually have much contrast?
6. In `6.sql`, write a SQL query to find the average entropy of prints by *Hiroshige*, rounded to two decimal places. Call the resulting column “Hiroshige Average Entropy”.
7. In `7.sql`, write a SQL query to list the English titles of the 5 brightest prints by *Hiroshige*, from most to least bright. Compare them to this [list on Wikipedia \(https://en.wikipedia.org/wiki/Thirty-six\\_Views\\_of\\_Mount\\_Fuji\\_\(Hiroshige\)\)](https://en.wikipedia.org/wiki/Thirty-six_Views_of_Mount_Fuji_(Hiroshige))) to see if your results match the print’s aesthetics.
8. In `8.sql`, write a SQL query to list the English titles of the 5 prints with the least contrast by *Hokusai*, from least to highest contrast. Compare them to this [list on Wikipedia \(https://en.wikipedia.org/wiki/Thirty-six\\_Views\\_of\\_Mount\\_Fuji\)](https://en.wikipedia.org/wiki/Thirty-six_Views_of_Mount_Fuji) to see if your results match the print’s aesthetics.
9. In `9.sql`, write a SQL query to find the English title and artist of the print with the highest brightness.
10. In `10.sql`, write a SQL query to answer a question of your choice about the prints. The query should:
  - Make use of `AS` to rename a column
  - Involve at least one condition, using `WHERE`
  - Sort by at least one column, using `ORDER BY`

## Usage

To test your queries as you write them in your `.sql` files, you can query the database by running

```
.read FILENAME
```

where `FILENAME` is the name of the file containing your SQL query. For example,

```
.read 1.sql
```

You can also run

```
$ cat FILENAME | sqlite3 views.db > output.txt
```

to redirect the output of the query to a text file called `output.txt`. (This can be useful for checking how many rows are returned by your query!)

## How to Test

While `check50` is available for this problem, you're encouraged to instead test your code on your own for each of the following. If you're using the `views.db` database provided in this problem's distribution, you should find that...

- Executing `1.sql` results in a table with 2 columns and 72 rows.
- Executing `2.sql` results in a table with 1 column and 3 rows.
- Executing `3.sql` results in a table with 1 column and 1 row.
- Executing `4.sql` results in a table with 1 column and 1 row.
- Executing `5.sql` results in a table with 1 column and 1 row.
- Executing `6.sql` results in a table with 1 column and 1 row.
- Executing `7.sql` results in a table with 1 column and 5 rows.
- Executing `8.sql` results in a table with 1 column and 5 rows.
- Executing `9.sql` results in a table with 2 columns and 1 row.

`10.sql` is up to you!

Note that row counts do not include header rows that only show column names.

## Correctness

```
check50 cs50/problems/2024/sql/views
```

## How to Submit

In your terminal, execute the below to submit your work.

```
submit50 cs50/problems/2024/sql/views
```

## Acknowledgements

Images and titles of prints retrieved from [en.wikipedia.org/wiki/Thirty-six\\_Views\\_of\\_Mount\\_Fuji](https://en.wikipedia.org/wiki/Thirty-six_Views_of_Mount_Fuji) ([https://en.wikipedia.org/wiki/Thirty-six\\_Views\\_of\\_Mount\\_Fuji](https://en.wikipedia.org/wiki/Thirty-six_Views_of_Mount_Fuji)) and [en.wikipedia.org/wiki/Thirty-six\\_Views\\_of\\_Mount\\_Fuji\\_\(Hiroshige\)](https://en.wikipedia.org/wiki/Thirty-six_Views_of_Mount_Fuji_(Hiroshige)) ([https://en.wikipedia.org/wiki/Thirty-six\\_Views\\_of\\_Mount\\_Fuji\\_\(Hiroshige\)](https://en.wikipedia.org/wiki/Thirty-six_Views_of_Mount_Fuji_(Hiroshige))).